

**SVEUČILIŠTE U ZAGREBU  
FAKULTET ORGANIZACIJE I INFORMATIKE  
VARAŽDIN**

**Zlatko Pračić**

# **API ZA RAD S RAZLIČITIM FORMATIMA DATOTEKA APACHE POI**

**SEMINAR  
NAPREDNE WEB TEHNOLOGIJE I SERVISI**

**Varaždin, 2026.**

**SVEUČILIŠTE U ZAGREBU**  
**FAKULTET ORGANIZACIJE I INFORMATIKE**  
**V A R A Ž D I N**

**Zlatko Pračić**

**Matični broj: 0016145097**

**Studij: Baze podataka i baze znanja**

**Kolegij: Napredne Web tehnologije i servisi**

**API ZA RAD S RAZLIČITIM FORMATIMA DATOTEKA APACHE POI**

**SEMINAR**

**Mentor:**

Prof. dr. sc. Dragutin Kermek

**Varaždin, lipanj 2026.**

**Izjava o izvornosti**

Izjavljujem kako je ovaj seminar izvorni rezultat mojeg rada te da se u izradi istoga nisam koristio drugim izvorima osim onima koji su u njemu navedeni. Za izradu rada su korištene etički prikladne i prihvatljive metode i tehnike rada.

*Autor potvrdio prihvatanjem odredbi u sustavu FOI Radovi*

---

## Sažetak

Apache POI je Java biblioteka otvorenog koda koju razvija Apache Software Foundation, a namijenjena je programskom čitanju, pisanju i izmjeni Microsoft Office dokumenata bez potrebe za instaliranim Microsoft Officeom. Biblioteka podržava kako starije binarne formate temeljene na OLE2 standardu (XLS, DOC, PPT), tako i moderne OOXML formate uvedene s Microsoft Officeom 2007 (XLSX, DOCX, PPTX). Rad prikazuje arhitekturu projekta i njegove ključne module, HSSF, XSSF, SXSSF, HWPF, XWPF, HSLF i XSLF te demonstrira praktičnu primjenu na dva konkretna primjera. Prvi primjer prikazuje programsko kreiranje Word dokumenta s tablicom podataka o studentima, uključujući stilizaciju zaglavlja i izmjenično bojanje redova, kao i tehniku cirkularnog pisma za masovnu personalizaciju dokumenata pomoću predloška i CSV izvora podataka. Drugi primjer prikazuje generiranje Excel izvještaja s primjenom stilova, ugrađenih formula i automatskog prilagođavanja širine stupaca. Zaključno se razmatraju prednosti i ograničenja biblioteke te smjernice za njezinu primjenu u poslovnim okruženjima.

**Ključne riječi:** Apache POI; Java; Microsoft Office; OOXML; Excel; Word; cirkularno pismo; grafovi; automatizacija; Maven

# Sadržaj

|                                                          |           |
|----------------------------------------------------------|-----------|
| <b>1. Uvod</b>                                           | <b>1</b>  |
| <b>2. Pregled Apache POI projekta</b>                    | <b>2</b>  |
| 2.1. Povijest i razvoj                                   | 2         |
| 2.2. Arhitektura i komponente                            | 2         |
| 2.3. Pregled dostupnih klasa                             | 3         |
| <b>3. Rad s formatima datoteka</b>                       | <b>4</b>  |
| 3.1. Excel datoteke                                      | 4         |
| 3.2. Word dokumenti                                      | 4         |
| 3.3. Ostale mogućnosti                                   | 5         |
| 3.3.1. PowerPoint prezentacije                           | 5         |
| 3.3.2. Enkripcija i dekripcija dokumenata                | 5         |
| 3.3.3. Ekstrakcija teksta                                | 5         |
| 3.3.4. Rad s metapodacima                                | 6         |
| 3.3.5. Podrška za Outlook i Visio                        | 6         |
| <b>4. Praktični primjeri</b>                             | <b>7</b>  |
| 4.1. Kreiranje Word dokumenta                            | 7         |
| 4.1.1. Zaglavlje i podnožje                              | 7         |
| 4.1.2. Paragrafi i stilovi                               | 8         |
| 4.1.3. Tablica                                           | 8         |
| 4.1.4. Spremanje dokumenta                               | 9         |
| 4.1.5. Izrada cirkularnog dokumenta                      | 10        |
| 4.2. Generiranje Excel izvještaja                        | 12        |
| 4.2.1. Stilovi i formatiranje                            | 12        |
| 4.2.2. Popunjavanje tablice                              | 12        |
| 4.2.3. Formule                                           | 13        |
| 4.2.4. Automatska širina stupaca i zamrzavanje zaglavlja | 13        |
| 4.2.5. Ugradnja grafa                                    | 15        |
| <b>5. Zaključak</b>                                      | <b>17</b> |
| <b>Popis isječaka koda</b>                               | <b>18</b> |
| <b>Popis literature</b>                                  | <b>19</b> |

|                                |           |
|--------------------------------|-----------|
| <b>Popis slika . . . . .</b>   | <b>20</b> |
| <b>Popis tablica . . . . .</b> | <b>21</b> |

# 1. Uvod

U suvremenom okruženju razmjena podataka putem Microsoft Office dokumenata je nešto što se podrazumijeva. Excel tablice, Word dokumenti i PowerPoint prezentacije postali su de-facto standard za izvještavanje, analizu podataka i komunikaciju. Ručna obrada takvih dokumenata u sustavima koji zahtijevaju automatizaciju, poput sustava za upravljanje sadržajem, poslovnih aplikacija ili alata za ekstrakciju podataka, nije praktična ni skalabilna.

Apache POI projekt, koji razvija i održava Apache Software Foundation, nudi rješenje za ovaj problem u obliku besplatne Java biblioteke otvorenog koda. Biblioteka programerima omogućuje čitanje, pisanje i izmjenu Microsoft Office datoteka isključivo korištenjem Java koda, bez potrebe za instaliranim Microsoft Officeom. Time se postiže potpuna platformska neovisnost, što je posebno vrijedno u poslovnim okruženjima koja se oslanjaju na Java ekosustav.

Motivacija za odabir Apache POI-a kao teme ovog rada leži upravo u njegovoj rasprostranjenosti i praktičnoj primjenjivosti. Biblioteka je jedna od najkorištenijih Java biblioteka za rad s Office formatima i aktivno se razvija već više od dva desetljeća. Podržava kako starije binarne formate temeljene na OLE2 standardu (XLS, DOC, PPT), tako i moderne XML-bazirane OOXML formate uvedene s Microsoft Officeom 2007 (XLSX, DOCX, PPTX).

Cilj ovog rada je sustavno prikazati arhitekturu i ključne komponente Apache POI projekta te demonstrirati njegovu primjenu na konkretnim primjerima. Rad počinje pregledom povijesti i arhitekture projekta, zatim obrađuje rad s pojedinim formatima, a završava praktičnim primjerima generiranja dokumenata.

## 2. Pregled Apache POI projekta

### 2.1. Povijest i razvoj

Začeci Apache POI projekta sežu u travanj 2001. godine, kada je programer Andrew Oliver dobio kratkoročni ugovor za generiranje Excel izvještaja u Javi. Suočen s drastičnim porastom cijene komercijalne biblioteke koju je dotad koristio, Oliver je odlučio razviti vlastito rješenje otvorenog koda. Uz pomoć Marca Johnsona, kojeg je pronašao putem lokalne Java korisničke grupe, implementirana je podrška za OLE 2 *Compound Document Format* kao temelj cijelog projekta. Johnson je preuzeo razvoj niskorazinskog sloja za rukovanje OLE2 strukturom, dok je Oliver paralelno radio na parsiranju XLS formata. Već s prvim izdanjem, POI 1.0, projekt je nadišao izvorne planove, uz podršku za čitanje i pisanje Excel datoteka, dodan je i serijalizacijski okvir za integraciju s drugim sustavima [1].

Kako je projekt rastao, postalo je jasno da nadilazi okvire jednog namjenskog alata. Nakon neuspjelog pokušaja integracije isključivo u Apache Cocoon projekt, donesena je odluka da se jezgri POI API-ji prenesu u okvir Apache Jakarta projekta, dok su pojedine komponente donirane drugim projektima iz Apache ekosustava. U sklopu Jakarte, projekt je nastavio rasti zahvaljujući doprinosima zajednice, proširujući podršku na nove formate i slučajeve korištenja. Početkom 2007. godine POI je zaprovodio proces inkubacije i postao samostalni projekt najviše razine unutar *Apache Software Foundation* [1].

Naziv projekta izvorno je bio akronim za “*Poor Obfuscation Implementation*”, humoristična referenca na činjenicu kako su Microsoftovi binarni formati, unatoč naizglednoj namjernoj kompliciranosti, bili uspješno reverzno razvijeni. Ista logika imenovanja proteže se i na potprojeke poput HSSF (*Horrible SpreadSheet Format*), HWPf (*Horrible Word Processor Format*) i ostale. Kako je projekt počeo privlačiti poslovne korisnike, objašnjenja akronima tiho su uklonjena s službenih stranica [2].

### 2.2. Arhitektura i komponente

Na ovoj infrastrukturi izgrađeni su moduli koji programerima nude intuitivan API za rad s Office dokumentima, apstrahirajući složenost internih binarnih i XML struktura. Pregled svih modula, njihovih formata i standarda prikazan je u Tablici 1, a detaljniji pregled ključnih klasa po kategorijama dan je u nastavku [3].



Tablica 1: Pregled Apache POI modula

| Modul     | Format                | Ekstenzija | Standard   |
|-----------|-----------------------|------------|------------|
| POIFS     | OLE2 datotečni sustav | ,          | OLE2       |
| HSSF      | Excel (stari)         | .xls       | OLE2       |
| XSSF      | Excel (novi)          | .xlsx      | OOXML      |
| SXSSF     | Excel (streaming)     | .xlsx      | OOXML      |
| HWPF      | Word (stari)          | .doc       | OLE2       |
| XWPF      | Word (novi)           | .docx      | OOXML      |
| HSLF      | PowerPoint (stari)    | .ppt       | OLE2       |
| XSLF      | PowerPoint (novi)     | .pptx      | OOXML      |
| HSMF      | Outlook poruke        | .msg       | OLE2       |
| HDGF/XDGF | Visio dijagrami       | .vsd/.vsdx | OLE2/OOXML |
| HPBF      | Publisher             | .pub       | OLE2       |

## 2.3. Pregled dostupnih klasa

Apache POI biblioteka pruža iznimno bogat skup klasa koje pokrivaju gotovo sve aspekte rada s Office dokumentima. Za Excel datoteke, klase poput `XSSFWorkbook`, `XSSFSheet`, `XSSFCell` i `XSSFCellStyle` omogućuju kreiranje i formatiranje tablica, dok `XSSFChart` i `XDDFChartData` podržavaju generiranje grafova. Za streaming obradu velikih datoteka dostupne su `SXSSFWorkbook` i `SXSSFSheet`. Word dokumenti pokriveni su klasama `XWPFRun`, `XWPFDocument`, `XWPFParagraph`, `XWPFTable` i `XWPFHeaderFooter`, a za PowerPoint prezentacije koriste se `XMLSlideShow`, `XSLFSlide`, `XSLFTextShape` i `XSLFTable` [4].

Uz rad s dokumentima, biblioteka nudi i infrastrukturne komponente. `POIFSFileSystem` i `OPCPackage` upravljaju niskorazinskim pristupom OLE2 i OOXML datotečnim strukturama. Klase iz paketa `EncryptionInfo`, `Encryptor` i `Decryptor` omogućuju enkripciju i dekripciju dokumenata. Za ekstrakciju teksta dostupne su `XWPFWordExtractor`, `ExcelExtractor` i `SlideShowExtractor`, a `DataFormatter` i `DateUtil` pomažu pri interpretaciji formata ćelija. Klase poput `EmbeddedExtractor` i `PictureData` omogućuju ekstrakciju ugrađenih slika i objekata iz dokumenata [4].

## 3. Rad s formatima datoteka

Ovo poglavlje rada posvećeno je praktičnoj primjeni Apache POI biblioteke za rad s tri najzastupljenija Microsoft Office formata, Excel tablicama, Word dokumentima i PowerPoint prezentacijama. Za svaki format postoje dvije implementacije koje odgovaraju dvjema generacijama Office standarda: starija binarna implementacija temeljena na OLE2 formatu te novija implementacija temeljena na OOXML standardu. U nastavku su opisane ključne mogućnosti svake implementacije, uz osvrt na specifičnosti pojedinih slučajeva korištenja [3].

### 3.1. Excel datoteke

Za rad s Excel datotekama Apache POI nudi tri implementacije. HSSF (*Horrible Spreadsheet Format*) pokriva stariji binarni .xls format koji odgovara Excel verzijama 97–2003, a XSSF (*XML Spreadsheet Format*) namijenjen je modernom .xlsx formatu temeljenom na OOXML standardu. Oba modula podržavaju čitanje i pisanje, a dostupna je i zajednička SS apstrakcija koja omogućuje pisanje koda neovisnog o konkretnom formatu. Isti kod može generirati i .xls i .xlsx datoteku promjenom jednog parametra. Kroz ove module moguće je kreirati radne knjige i listove, postavljati vrijednosti ćelija, primjenjivati stilove i formate, koristiti formule te upravljati uvjetnim oblikovanjem [5].

Za slučajeve kada je potrebno generirati izrazito velike Excel datoteke, standardni XSSF pristup može biti problematičan zbog visokog memorijskog opterećenja, cijela struktura dokumenta drži se u memoriji. Kao rješenje, od verzije 3.8-beta3 dostupan je SXSSF (*Streaming Spreadsheet Format*), streaming implementacija izgrađena povrh XSSF-a. SXSSF postiže mali memorijski otisak ograničavanjem pristupa samo redovima unutar kliznog prozora fiksne veličine, dok se stariji redovi automatski zapisuju na disk. Veličina prozora može se konfigurirati pri inicijalizaciji radne knjige ili zasebno po listu. Važno je napomenuti kako je SXSSF namijenjen isključivo pisanju, za čitanje velikih datoteka preporučuje se korištenje SAX-temeljenog Event modela koji XML strukturu .xlsx datoteke procesira inkrementalno [5].

### 3.2. Word dokumenti

Za rad s Word dokumentima Apache POI nudi dvije implementacije. HWPf (*Horrible Word Processor Format*) pokriva stariji binarni .doc format koji odgovara Word verzijama 97–2003, a XWPF (*XML Word Processor Format*) namijenjen je modernom .docx formatu temeljenom na OOXML standardu. HWPf podržava čitanje i ograničeno pisanje, a pruža i jednostavnu ekstrakciju teksta za još starije Word 6 i Word 95 formate. XWPF nudi stabilniji i potpuniji API te podržava čitanje i pisanje složenijih dokumenata [6].

Glavna ulazna točka za rad s .docx datotekama je klasa XWPFDocument, iz koje se dohvaćaju svi elementi dokumenta. Paragrafi su dostupni kroz klasu XWPFParagraph, a svaki paragraf sastoji se od jednog ili više XWPFRun objekata koji predstavljaju tekstualne odsječke s jednakim stilskim svojstvima: fontom, veličinom, podebljavanjem, nakošenje slova

i slično. Tablice se dohvaćaju kroz klasu `XWPFTable`, pri čemu su pojedini redovi i ćelije dostupni kroz `XWPFTableRow` i `XWPFTableCell`. Za ekstrakciju cjelokupnog teksta dokumenta u jednom koraku dostupna je klasa `XWPFFWordExtractor`. Zaglavlja i podnožja dostupna su kroz `XWPFHeader` i `XWPFHeaderFooterPolicy` klase putem `XWPFHeaderFooterPolicy` objekta [6].

### 3.3. Ostale mogućnosti

Uz osnovnu podršku za Excel i Word, Apache POI biblioteka nudi niz dodatnih mogućnosti koje pokrivaju specifične poslovne i tehničke potrebe.

#### 3.3.1. PowerPoint prezentacije

Kroz XSLF API moguće je iterirati po svim oblicima na svakom slajdu i ekstrahirati tekstualni sadržaj, što je korisno za indeksiranje prezentacija. Metoda `draw()` klase `XSLFSlide` omogućuje iscrtavanje cijelog slajda na `java.awt.Graphics2D` objekt, čime je moguće generirati PNG ili JPEG slike pojedinih slajdova bez instaliranog PowerPointa. Ova mogućnost koristi se u sustavima za pregled prezentacija u web preglednicima, gdje je potrebno prikazati slajdove bez instaliranog Microsoft Officea na poslužitelju. Uz to, XSLF API podržava dodavanje slika, tablica i tekstualnih okvira na slajdove, postavljanje tranzicija te upravljanje slide masterom i layoutima [7].

#### 3.3.2. Enkripcija i dekripcija dokumenata

Apache POI podržava enkripciju i dekripciju Office dokumenata standardnim OOXML mehanizmom zaštite lozinkom. Enkripcija se provodi kroz klase `EncryptionInfo` i `Encryptor`, pri čemu se dokument zapisuje u `POIFSFileSystem` kontejner koji je šifriran odabranim algoritmom. Podržani algoritmi uključuju AES-128 i AES-256 koji se koriste u modernim verzijama Microsoft Officea. Dekripcija se provodi analognim postupkom kroz klasu `Decryptor` — dokument se učitava kao `POIFSFileSystem`, provjerava se ispravnost lozinke i dohvaća dešifrirani tok podataka iz kojeg se rekonstruira radna knjiga ili dokument [4].

#### 3.3.3. Ekstrakcija teksta

Za slučajeve kada nije potrebna puna manipulacija dokumentom nego samo dohvat tekstualnog sadržaja, Apache POI nudi specijalizirane klase za ekstrakciju teksta. `XWPFFWordExtractor` ekstrahira sav tekst iz `.docx` datoteke uključujući paragrafe, tablice i zaglavlja, `XSSFExcelExtractor` radi analogno za `.xlsx` datoteke, a `SlideShowExtractor` za `.pptx` prezentacije. Sve tri klase implementiraju zajednički `POITextExtractor` sučelje, što omogućuje uniforman pristup ekstrakciji bez obzira na format dokumenta. Ekstrakcija teksta posebno je korisna u sustavima za pretraživanje i indeksiranje sadržaja, kao što su Apache Lucene i Apache Solr, s kojima Apache POI ima izravnu integraciju putem Apache Tika biblioteke [4].

### 3.3.4. Rad s metapodacima

Svaki Office dokument sadrži metapodatke poput naslova, autora, datuma kreiranja i izmjene te ključnih riječi. Apache POI omogućuje čitanje i pisanje tih metapodataka kroz `POIXMLProperties` klasu za OOXML formate, koja sadrži tri podskupa svojstava: osnovna svojstva (`CoreProperties`), proširena svojstva (`ExtendedProperties`) i prilagođena svojstva (`CustomProperties`). Za starije OLE2 formate metapodaci su dostupni kroz `HPSF` komponentu (*Horrible Property Set Format*). Ova mogućnost korisna je u sustavima za upravljanje dokumentima gdje je potrebno automatski popunjavati ili čitati metapodatke bez otvaranja samog dokumenta [4].

### 3.3.5. Podrška za Outlook i Visio

Apache POI uključuje i module za formate koji su rjeđe zastupljeni u tipičnim poslovnim okruženjima. `HSMF` (*Horrible Stupid Mail Format*) modul omogućuje čitanje Outlook `.msg` datoteka, pri čemu su dostupni pošiljatelj, primatelji, predmet, tijelo poruke i privici. Microsoft je 2007. godine otvorio specifikaciju ovog formata, što je dodatno olakšalo razvoj pouzdane implementacije [3].

Za Visio dijagrame dostupna su dva modula: `HDGF` pokriva stariji binarni `.vsd` format i podržava samo niskorazinsko čitanje i ekstrakciju teksta, dok `XDGF` pokriva moderniji XML-bazirani `.vsdx` format s nešto širom podrškom. Oba modula trenutno ne nude API za kreiranje novih Visio dijagrama programskim putem, već su usmjereni primarno na čitanje i ekstrakciju sadržaja [3].

## 4. Praktični primjeri

Za demonstraciju praktične primjene Apache POI biblioteke koristi se Eclipse IDE uz Maven kao alat za upravljanje ovisnostima. Maven omogućuje jednostavno upravljanje vanjskim bibliotekama kroz centralnu konfiguracijsku datoteku `pom.xml`, čime se izbjegava ručno preuzimanje i dodavanje JAR datoteka u classpath projekta [8].

Nakon kreiranja novog Maven projekta u Eclipseu putem `File → New → Maven Project`, u datoteku `pom.xml` potrebno je dodati dvije ovisnosti. Ovisnost `poi` pokriva podršku za starije binarne formate te predstavlja jezgrenu biblioteku projekta, dok ovisnost `poi-ooxml` dodaje podršku za moderne OOXML formate. Maven automatski preuzima sve tranzitivne ovisnosti, pa nije potrebno ručno upravljati dodatnim JAR datotekama. Trenutno aktualna stabilna verzija Apache POI biblioteke je 5.5.1. [8]

```
1 <dependency>
2     <groupId>org.apache.poi</groupId>
3     <artifactId>poi</artifactId>
4     <version>5.5.1</version>
5 </dependency>
6 <dependency>
7     <groupId>org.apache.poi</groupId>
8     <artifactId>poi-ooxml</artifactId>
9     <version>5.5.1</version>
10 </dependency>
```

---

Isječak koda 1: Ovisnosti u datoteci `pom.xml`

### 4.1. Kreiranje Word dokumenta

Kreiranje Word dokumenta u Apache POI-u temelji se na klasi `XWPFDocument`, koja predstavlja glavni objekt dokumenta. Iz nje se kreiraju svi ostali elementi, zaglavlja, podnožja, paragrafi i tablice [9].

#### 4.1.1. Zaglavlje i podnožje

Zaglavlje i podnožje kreiraju se metodama `createHeader()` i `createFooter()` uz parametar `HeaderFooterType.DEFAULT`, koji označava da se primjenjuju na sve stranice dokumenta. Unutar zaglavlja i podnožja kreira se paragraf, a tekst se dodaje kroz `XWPFRun` objekt. Isti princip, paragraf koji sadrži jedan ili više `XWPFRun` objekata, koristi se za sve tekstualne elemente u dokumentu [9].

```
1 XWPFHeader zaglavlje = dokument.createHeader(HeaderFooterType.DEFAULT);
2 XWPFParagraph zaglavljeParagraf = zaglavlje.createParagraph();
3 zaglavljeParagraf.setAlignment(ParagraphAlignment.CENTER);
4 XWPFRun zaglavljeRun = zaglavljeParagraf.createRun();
5 zaglavljeRun.setText("Fakultet organizacije i informatike");
```

```

6 zaglavljeRun.setFontSize(10);
7 zaglavljeRun.setColor("888888");

```

---

Isječak koda 2: Obrada Word dokumenta

### 4.1.2. Paragrafi i stilovi

Paragrafi se kreiraju metodom `createParagraph()` na objektu dokumenta. Svaki paragraf može imati postavljeno poravnanje (`setAlignment()`), razmak prije ili nakon (`setSpacingBefore()`, `setSpacingAfter()`) te jedan ili više `XWPFRun` objekata koji nose stvarni tekst. Na razini `XWPFRun` objekta definiraju se stilska svojstva teksta: font (`setFontFamily()`), veličina (`setFontSize()`), boja (`setColor()`), podebljanje (`setBold()`) i kurziv (`setItalic()`). U primjeru su na ovaj način kreirani naslov, podnaslov, uvodni paragraf i završna napomena [9].

```

1 XWPFParagraph naslov = dokument.createParagraph();
2 naslov.setAlignment(ParagraphAlignment.CENTER);
3 naslov.setSpacingAfter(200);
4 XWPFRun naslovRun = naslov.createRun();
5 naslovRun.setText("Popis studenata i ocjena");
6 naslovRun.setBold(true);
7 naslovRun.setFontSize(20);
8 naslovRun.setFontFamily("Calibri");

```

---

Isječak koda 3: Kreiranje naslova

### 4.1.3. Tablica

Tablica se kreira metodom `createTable()` uz navođenje broja redova i stupaca. U primjeru je kreirana tablica s 6 redova i 3 stupca, prvi red služi kao zaglavlje tablice, a preostalih 5 redova sadrži podatke o studentima [9].

Zaglavlje tablice stilizira se plavom pozadinskom bojom (`setColor()`) i bijelim podebljanim tekstom. Svaka ćelija zaglavlja dohvaća se metodom `getCell()` iz odgovarajućeg reda, a tekst se dodaje na isti način kao i u paragrafima dokumenta, kroz `XWPFParagraph` i `XWPFRun` objekte [9].

```

1 XWPFTable tablica = dokument.createTable(6, 3);
2 tablica.setWidth("100%");
3
4 String[] zaglavljaStupaca = {"Ime i prezime", "JMBAG", "Ocjena"};
5 XWPFTableRow zaglavljeRed = tablica.getRow(0);
6 for (int i = 0; i < zaglavljaStupaca.length; i++) {
7     XWPFTableCell celija = zaglavljeRed.getCell(i);
8     celija.setColor("2E75B6");
9     XWPFParagraph p = celija.getParagraphs().get(0);
10    p.setAlignment(ParagraphAlignment.CENTER);

```

```

11     XWPFRun r = p.createRun();
12     r.setText(zaglavljajaStupaca[i]);
13     r.setBold(true);
14     r.setColor("FFFFFF");
15     r.setFontSize(11);
16 }

```

---

#### Isječak koda 4: Kreiranje zaglavlja tablice

Redovi s podacima pune se iteracijom kroz dvodimenzionalno polje. Radi bolje čitljivosti, primijenjeno je izmjenično bojanje redova, parni redovi ostaju bijeli, a neparni dobivaju svijetloplavu pozadinsku boju [9].

```

1  for (int i = 0; i < podaci.length; i++) {
2      XWPFTableRow red = tablica.getRow(i + 1);
3      String boja = (i % 2 == 0) ? "FFFFFF" : "DEEAF1";
4      for (int j = 0; j < podaci[i].length; j++) {
5          XWPFTableCell celija = red.getCell(j);
6          celija.setColor(boja);
7          XWPFParagraph p = celija.getParagraphs().get(0);
8          p.setAlignment(ParagraphAlignment.CENTER);
9          XWPFRun r = p.createRun();
10         r.setText(podaci[i][j]);
11         r.setFontSize(11);
12     }
13 }

```

---

#### Isječak koda 5: Punjenje tablice podacima

### 4.1.4. Spremanje dokumenta

Po završetku izgradnje dokumenta, korištenjem `FileOutputStream` sadržaj se zapisuje u datoteku objekta. Preporučuje se korištenje `try-with-resources` bloka koji osigurava automatsko zatvaranje toka podataka nakon zapisivanja, čime se sprječava curenje resursa [10].

```

1  try (FileOutputStream out = new FileOutputStream("studenti.docx")) {
2      dokument.write(out);
3  }
4  dokument.close();

```

---

#### Isječak koda 6: Spremanje dokumenta

Na Slici 1 prikazan je finalni izgled dokumenta u formatu `.docx` generiranog programskim putem. U gornjem dijelu dokumenta vidljivo je zaglavlje (*header*) s centriranim tekstom institucije, dok je u središnjem dijelu naslov definiran povećanim fontom i podebljanim stilom (**bold**), čime se postiže jasna vizualna hijerarhija. Ispod naslova nalazi se tekstualni odlomak koji služi kao uvod u podatke.

Središnji strukturni element dokumenta je tablica s podacima o studentima. Na slici se vidi primjena stilova nad tablicom:

- **Zaglavlje tablice** istaknuto je tamnijom bojom pozadine i bijelim tekstom radi bolje čitljivosti.
- **Tijelo tablice** koristi tehniku izmjeničnog bojanja redova (tzv. *zebra striping*), što olakšava praćenje podataka u redovima.
- **Poravnanje sadržaja** unutar ćelija je uniforman.

Na dnu stranice nalazi se završna napomena, a cijeli dokument zadržava standardne margine i tipografiju karakterističnu za moderne Microsoft Word dokumente, iako je u potpunosti generiran bez korištenja samog Microsoft Office sučelja.

Fakultet organizacije i informatike - Varaždin

## Popis studenata i ocjena

Akadska godina 2025./2026.

U nastavku se nalazi popis studenata s postignutim ocjenama na kolegiju Napredne WEB tehnologije i servisi. Tablica prikazuje ime i prezime studenta, JMBAG te završnu ocjenu.

| Ime i prezime  | JMBAG      | Ocjena         |
|----------------|------------|----------------|
| Emanuel Pračić | 0123456789 | Izvrstan (5)   |
| Pero Kos       | 0123456789 | Izvrstan (5)   |
| Franjo Kovač   | 0123456790 | Vrlo dobar (4) |
| Maja Perić     | 0123456791 | Izvrstan (5)   |
| Luka Novak     | 0123456792 | Dobar (3)      |
| Sara Babić     | 0123456793 | Vrlo dobar (4) |

Dokument generiran programski korištenjem Apache POI biblioteke.

Slika 1: Word dokument kreiran pomoću Apache POI biblioteke

### 4.1.5. Izrada cirkularnog dokumenta

Na ovom mjestu opisuje se ideja izgradnje cirkularnog pisma uz pomoć `XWPFDocument` klase. Cirkularno pismo je tehnika masovne personalizacije dokumenta. Prije nego što se uopće krene s implementacijom, potrebno je kreirati predložak dokumenta koji sadrži rezervirana mjesta (placeholderi) za dinamičke podatke. U konkretnom primjeru, u Word dokumentu se koriste oznake `{{IME}}`, `{{PREZIME}}`, `{{JMBAG}}`, `{{KOLEGIJ}}`, `{{DATUM}}` i `{{OCJENA}}` koje će se kasnije zamijeniti stvarnim vrijednostima iz CSV datoteke kao izvora podataka. Ove oznake su lako prepoznatljive i ne sadrže razmake, što olakšava njihovu identifikaciju u kodu.

Predložak bi izgledao na sljedeći način:



Poštovani/a {{IME}} {{PREZIME}},

na kolegiju {{KOLEGIJ}} ste ostvarili ocjenu {{OCJENA}}.

JMBAG: {{JMBAG}}

Datum: {{DATUM}}

S poštovanjem,

Fakultet organizacije i informatike

Za potrebu popune navedenog predloška, koristi se CSV datoteka koja sadrži podatke o studentima i njihovim ocjenama. Ova datoteka služi kao izvor podataka za zamjenu rezerviranih mjesta u predlošku, omogućujući generiranje personaliziranih dokumenata za svakog studenta.

```
1 ime, prezime, jmbag, kolegij, ocjena
2 Pero, Kos, 0123456789, Napredne WEB tehnologije i servisi, Izvrstan (5)
3 Franjo, Kovač, 0123456790, Napredne WEB tehnologije i servisi, Vrlo dobar (4)
4 Maja, Perić, 0123456791, Napredne WEB tehnologije i servisi, Izvrstan (5)
5 Luka, Novak, 0123456792, Napredne WEB tehnologije i servisi, Dobar (3)
6 Sara, Babić, 0123456793, Napredne WEB tehnologije i servisi, Vrlo dobar (4)
```

#### Isječak koda 7: Datoteka studenti.csv

U Prilogu se nalazi Java kod koji demonstrira kako se koristi Apache POI za čitanje predloška, zamjenu rezerviranih mjesta s podacima iz CSV datoteke i generiranje personaliziranih Word dokumenata za svakog studenta. Ovaj primjer ilustrira praktičnu primjenu Apache POI biblioteke u kontekstu masovne personalizacije dokumenata, što je česta potreba u obrazovnim institucijama i poslovnim okruženjima.

Program se sastoji od dvije metode: `main()` koja upravlja ukupnim tokom izvođenja i pomoćne metode `zamijeniUParagrafu()` koja provodi zamjenu placeholdera unutar pojedinog paragrafa.

U metodi `main()` najprije se definiraju putanje do predloška i CSV datoteke te se automatski generira trenutni datum korištenjem `LocalDate.now()`. CSV datoteka čita se redak po redak korištenjem `BufferedReader` klase, pri čemu se prvi redak (zaglavlje) preskače zastavicom `prviRed`. Svaki sljedeći redak dijeli se po zarezu metodom `split(",")`, a vrijednosti se pohranjuju u zasebne varijable. Za svakog studenta predložak se učitava kao novi `XWPFDocument` objekt kako bi svaki generirani dokument krenuo od čistog predloška. Po završetku zamjene, dokument se sprema pod nazivom oblika `obavijest_Ime_Prezime.docx`.

Metoda `zamijeniUParagrafu()` rješava specifičan problem XWPF implementacije, Word dokument interno može razbiti jedan vidljivi tekst u više `XWPFRun` objekata, zbog čega jednostavna zamjena po pojedinom odsječku ne bi mogla prepoznati cijeli placeholder. Metoda najprije spaja tekstove svih odsječaka paragrafa u jedan string korištenjem `StringBuilder` klase, zatim provodi zamjenu svih placeholdera, a konačni tekst upisuje u prvi odsječak dok ostale odsječke prazni. Na taj način stilska svojstva prvog odsječka ostaju sačuvana, a sadržaj je ispravno popunjen.

## 4.2. Generiranje Excel izvještaja

Generiranje Excel datoteke u Apache POI-u temelji se na hijerarhiji triju objekata: `XSSFWorkbook` predstavlja cijelu radnu knjigu (engl. *workbook*), `XSSFSheet` predstavlja jedan list unutar nje, a `Row` i `Cell` odgovaraju retku i ćeliji na listu. Svi se kreiraju redom, od radne knjige prema ćeliji [11].

### 4.2.1. Stilovi i formatiranje

Za razliku od rada s Word dokumentima gdje se stil veže uz `XWPFRun` objekt, u Excel modulima stil se definira kao zasebni `XSSFCellStyle` objekt koji se kreira na razini radne knjige i potom dodjeljuje pojedinim ćelijama. Jedan stil može se primijeniti na više ćelija, što je važno jer svaka radna knjiga podržava ograničen broj jedinstvenih stilova. Font se kreira odvojeno metodom `createFont()` na objektu radne knjige te potom veže uz stil metodom `setFont()` [11].

Boja pozadine ćelije postavlja se kombinacijom:

- `setFillForegroundColor()`
- `setFillPattern()`

pri čemu se za jednobožno ispunjenje koristi `FillPatternType.SOLID_FOREGROUND`. Boja se predaje kao `XSSFCOLOR` objekt koji prima standardni `java.awt.Color`:

```
1 XSSFCellStyle stilZaglavlja = radnaKnjiga.createCellStyle();
2 stilZaglavlja.setFillForegroundColor(new XSSFCOLOR(
3     new java.awt.Color(46, 117, 182), new DefaultIndexedColorMap()));
4 stilZaglavlja.setFillPattern(FillPatternType.SOLID_FOREGROUND);
5 stilZaglavlja.setAlignment(HorizontalAlignment.CENTER);
6
7 XSSFFont fontZaglavlja = radnaKnjiga.createFont();
8 fontZaglavlja.setBold(true);
9 fontZaglavlja.setColor(new XSSFCOLOR(
10     new java.awt.Color(255, 255, 255), new DefaultIndexedColorMap()));
11 fontZaglavlja.setFontHeightInPoints((short) 11);
12 stilZaglavlja.setFont(fontZaglavlja);
```

---

Isječak koda 8: Kreiranje stilova ćelija

### 4.2.2. Popunjavanje tablice

Redovi se kreiraju metodom `createRow()` na objektu lista, a metodom `createCell()` ćelije na retku. Vrijednost ćelije postavlja se metodom `setCellValue()`, koja je preopterećena i prima `String`, `double`, `boolean` ili `Date`. Numeričke vrijednosti potrebno je pohraniti kao broj (a ne kao string) kako bi formule mogle nad njima raditi ispravno [11].

U primjeru je primijenjeno isto izmjenično bojanje redova kao u Word tablici iz prethodne sekcije: parni redovi ostaju bijeli, a neparni dobivaju svijetloplavu pozadinsku boju.

```
1 for (int i = 0; i < podaci.length; i++) {
2     Row red = list.createRow(i + 1);
3     XSSFCellStyle stil = (i % 2 == 0) ? stilParni : stilNeparni;
4     for (int j = 0; j < podaci[i].length; j++) {
5         Cell celija = red.createCell(j);
6         if (j == 3) {
7             celija.setCellValue(Integer.parseInt(podaci[i][j]));
8         } else {
9             celija.setCellValue(podaci[i][j]);
10        }
11        celija.setCellStyle(stil);
12    }
13 }
```

---

Isječak koda 9: Popunjavanje tablice podacima

### 4.2.3. Formule

Formule se unose metodom `setCellFormula()` koja prima string s formulom u Excel sintaksi, identičnoj onoj kakvu korisnik upisuje izravno u Excel. U primjeru se koristi formula `AVERAGE(D2:D6)` koja izračunava prosjek numeričkih ocjena unesenih u stupac D [11].

```
1 Row redFormule = list.createRow(zadnjiRed);
2
3 Cell celijaOznaka = redFormule.createCell(2);
4 celijaOznaka.setCellValue("Prosjek ocjena:");
5 celijaOznaka.setCellStyle(stilZbroj);
6
7 Cell celijaFormula = redFormule.createCell(3);
8 celijaFormula.setCellFormula("AVERAGE(D2:D6)");
9 celijaFormula.setCellStyle(stilZbroj);
```

---

Isječak koda 10: Unos formule i formatiranje retka

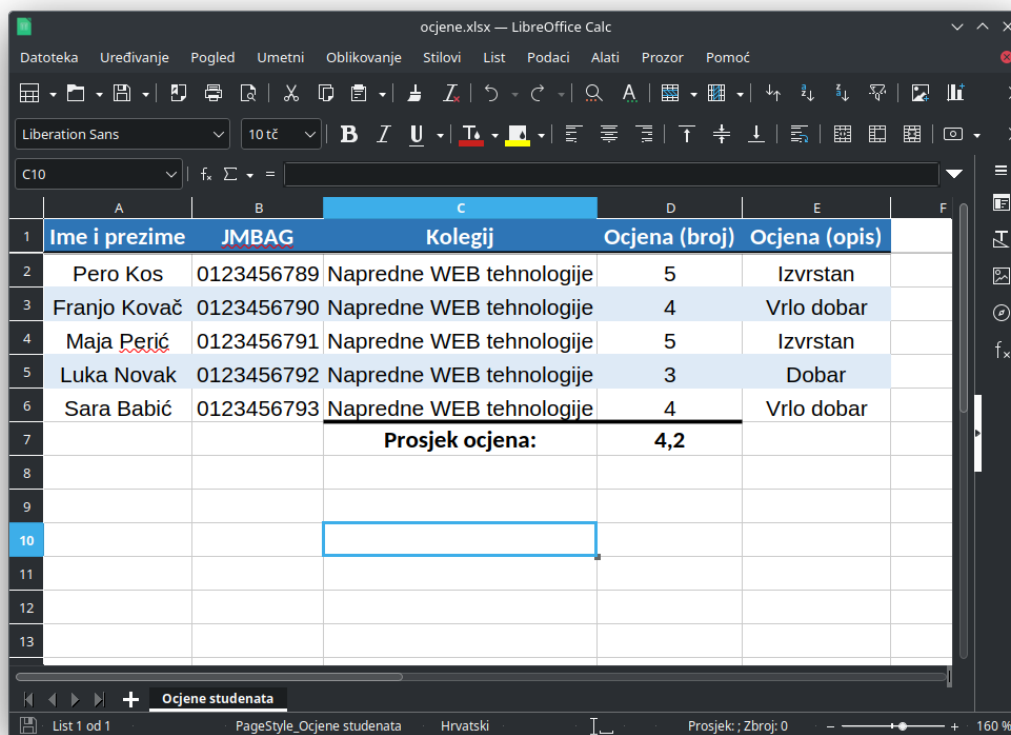
### 4.2.4. Automatska širina stupaca i zamrzavanje zaglavlja

Apache POI nudi dvije korisne metode za poboljšanje preglednosti tablice. Metoda `autoSizeColumn()` automatski prilagođava širinu stupca najširem sadržaju u njemu, a zamrzava određenog broja redova ili stupaca metodom `createFreezePane()`, čime zaglavlje ostaje vidljivo pri pomicanju kroz dugačke tablice [11].

```
1 for (int i = 0; i < stupci.length; i++) {
2     list.autoSizeColumn(i);
3 }
4 list.createFreezePane(0, 1);
```

## Isječak koda 11: Automatska širina i zamrzavanje zaglavlja

Na Slici 2 prikazan je finalni izgled datoteke u formatu `.xlsx` generirane programskim putem.



| Ime i prezime    | JMBAG      | Kolegij                  | Ocjena (broj) | Ocjena (opis) |
|------------------|------------|--------------------------|---------------|---------------|
| Pero Kos         | 0123456789 | Napredne WEB tehnologije | 5             | Izvrstan      |
| Franjo Kovač     | 0123456790 | Napredne WEB tehnologije | 4             | Vrlo dobar    |
| Maja Perić       | 0123456791 | Napredne WEB tehnologije | 5             | Izvrstan      |
| Luka Novak       | 0123456792 | Napredne WEB tehnologije | 3             | Dobar         |
| Sara Babić       | 0123456793 | Napredne WEB tehnologije | 4             | Vrlo dobar    |
| Prosjeck ocjena: |            |                          | 4,2           |               |
|                  |            |                          |               |               |
|                  |            |                          |               |               |
|                  |            |                          |               |               |
|                  |            |                          |               |               |
|                  |            |                          |               |               |
|                  |            |                          |               |               |

Slika 2: Excel dokument kreiran pomoću Apache POI biblioteke

U donjem dijelu sučelja vidljiva je kartica lista s nazivom *Ocjene studenata*, koji je postavljen pri kreiranju lista metodom `createSheet()`.

Središnji strukturni element je tablica s podacima o studentima. Na slici se vidi primjena stilova:

- **Zaglavlje tablice** (redak 1) istaknuto je plavom pozadinom i bijelim podebljanim tekstom, uz automatski prilagođenu širinu stupaca metodom `autoSizeColumn()`.
- **Tijelo tablice** koristi tehniku izmjeničnog bojanja redova (tzv. *zebra striping*), što olakšava praćenje podataka u redovima.
- **Redak s formulom** (redak 7) vizualno je odvojen gornjim rubom i podebljanim tekstom, a ćelija D7 prikazuje rezultat formule `AVERAGE(D2:D6)`, izračunati prosjek ocjena iznosi 4,2.

## 4.2.5. Ugradnja grafa

Uz tablične podatke, Apache POI omogućuje programsko kreiranje grafova koji su izravno vezani uz ćelije lista. Graf se kreira kroz `XSSFDrawing` objekt koji predstavlja sloj za crtanje na listu, a njegova pozicija i veličina definiraju se kroz `XSSFClientAnchor` parametar koji prima koordinate u obliku indeksa stupaca i redova [11].

```
1 XSSFDrawing drawing = list.createDrawingPatriarch();
2 XSSFClientAnchor anchor = drawing.createAnchor(0, 0, 0, 0, 0, 8, 6, 25);
3
4 XSSFChart chart = drawing.createChart(anchor);
5 chart.setTitleText("Ocjene studenata");
6 chart.setTitleOverlay(false);
7
8 XDDFChartLegend legenda = chart.getOrAddLegend();
9 legenda.setPosition(LegendPosition.BOTTOM);
10
11 XDDFCategoryAxis xOs = chart.createCategoryAxis(AxisPosition.BOTTOM);
12 xOs.setTitle("Student");
13 XDDFValueAxis yOs = chart.createValueAxis(AxisPosition.LEFT);
14 yOs.setTitle("Ocjena");
15 yOs.setCrosses(AxisCrosses.AUTO_ZERO);
16
17 XDDFDataSource<String> imena = XDDFDataSourcesFactory.fromStringCellRange(
18     list, new CellRangeAddress(1, 5, 0, 0));
19 XDDFNumericalDataSource<Double> ocjene = XDDFDataSourcesFactory
20     .fromNumericCellRange(list, new CellRangeAddress(1, 5, 3, 3));
21
22 XDDFChartData podaciGrafa = chart.createData(ChartTypes.BAR, xOs, yOs);
23 podaciGrafa.setVaryColors(false);
24 XDDFChartData.Series serija = podaciGrafa.addSeries(imena, ocjene);
25 serija.setTitle("Ocjena", null);
26 ((XDDFBarChartData) podaciGrafa).setBarDirection(BarDirection.COL);
27
28 chart.plot(podaciGrafa);
```

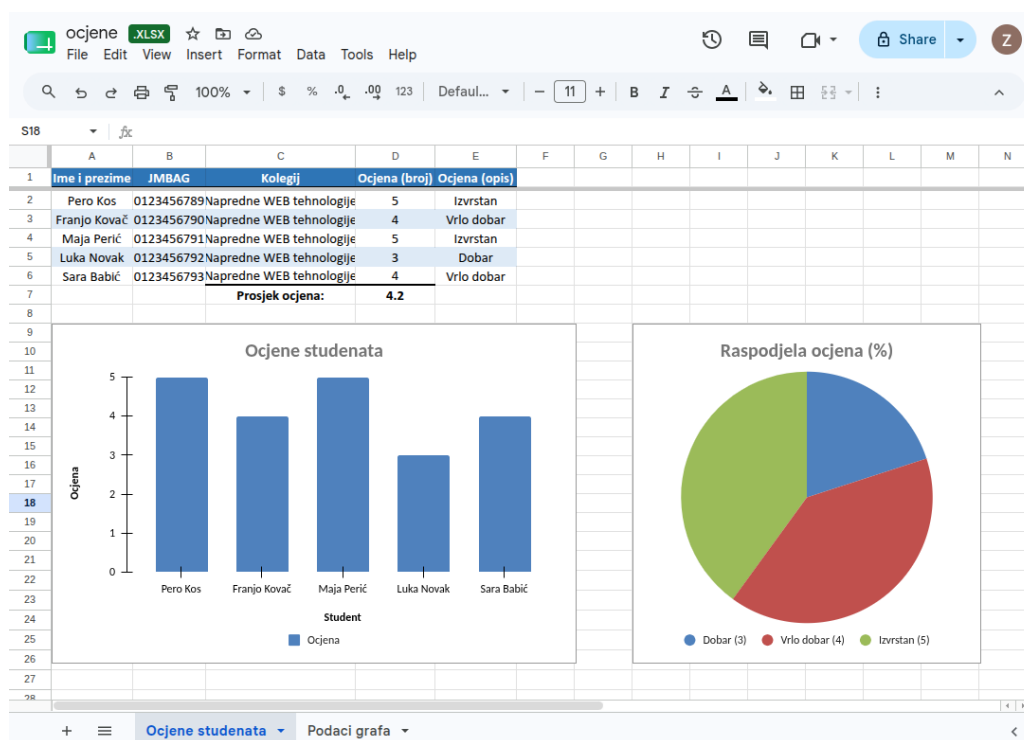
---

Isječak koda 12: Kreiranje stupčastog grafa

`S XDDFDataSourcesFactory.fromStringCellRange()` i `fromNumericCellRange()` metodama se definiraju izvori podataka za graf. Iste primaju referencu na list i raspon ćelija kroz `CellRangeAddress` objekt. U primjeru raspon `A2:A6` sadrži imena studenata kao kategorijsku os, a raspon `D2:D6` sadrži numeričke ocjene kao vrijednosnu os. Ključna prednost ovog pristupa je što graf nije statična slika nego živi objekt vezan uz podatke u ćelijama, pri svakom otvaranju datoteke u Excelu graf se automatski ažurira ako su se podaci promijenili.

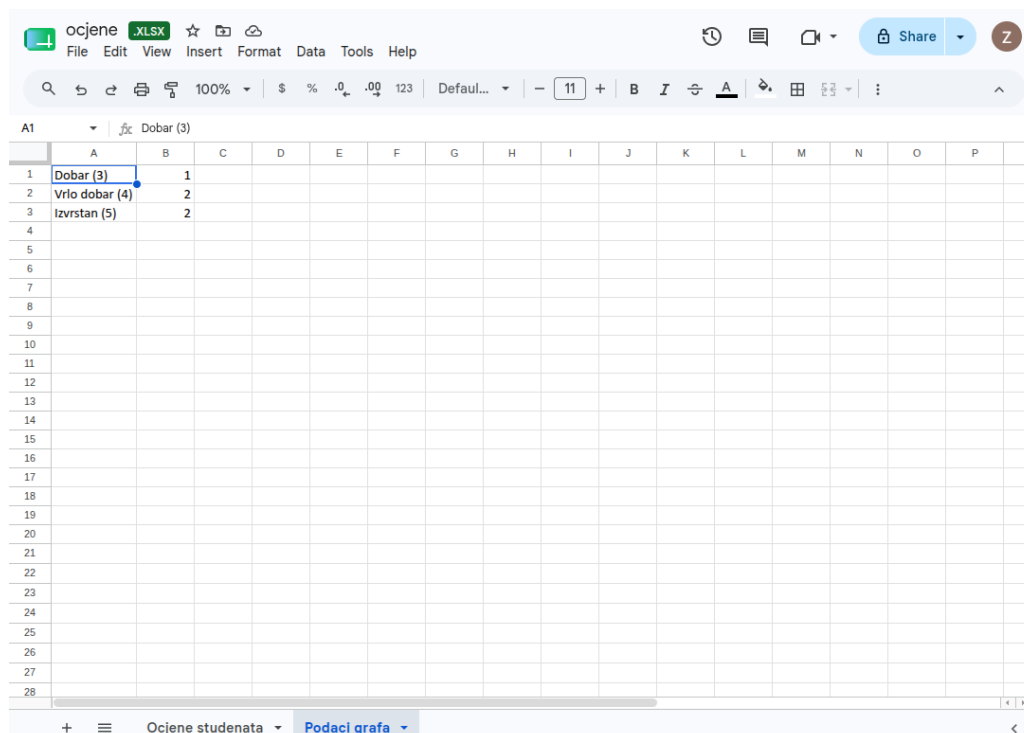
Na Slici 3 prikazan je finalni izgled datoteke u formatu `.xlsx` generirane programskim putem. Prikaz je preuzet s Google Sheets aplikacije, ali isti graf ispravno se prikazuje i u Microsoft Excelu. LibreOffice Calc ne renderira stupce grafa ispravno te prikazuje samo prazne osi, što je ograničenje kompatibilnosti između POI-jevog XDDF API-ja i LibreOfficeovog parsera

OOXML grafova.



Slika 3: Excel dokument kreiran pomoću Apache POI biblioteke

Na Slici 4 je prikazan list dokumenta gdje se kreirao skup podataka potreban za drugi graf (Py chart).



Slika 4: Excel dokument kreiran pomoću Apache POI biblioteke

## 5. Zaključak

Apache POI pokazao se kao pouzdana biblioteka za programski rad s Microsoft Office formatima u Java okruženju. Kroz praktične primjere u ovom radu vidljivo je kako generiranje strukturiranih Word i Excel dokumenata je izvedivo s relativno malo koda, uz punu kontrolu nad stilovima, sadržajem i formatiranjem. Najveća prednost biblioteke je platformska neovisnost, dokumenti se generiraju bez potrebe za instaliranim Microsoft Officeom, što je ključno za serverske i automatizacijske scenarije. Podrška za oba standarda, OLE2 i OOXML, osigurava kompatibilnost s dokumentima različitih generacija.

Ograničenja su ipak prisutna. SXSSF streaming način rada ograničen je isključivo na pisanje, a čitanje velikih datoteka zahtijeva SAX-temeljeni pristup koji je znatno složeniji za implementaciju. HSLF i XSLF moduli za PowerPoint nemaju zajedničko sučelje, što otežava pisanje koda neovisnog o formatu. Uz to, Word dokument interno može razbiti jedan vidljivi tekst u više XWPFRun objekata, što je potrebno uzeti u obzir pri implementaciji zamjene teksta, kao što je prikazano u primjeru cirkularnog pisma.

Najveći izazov pri radu s Apache POI bibliotekom bio je upravo problem razbijanja teksta na više XWPFRun objekata, što se pokazalo tek pri testiranju cirkularnog pisma kada zamjena placeholdera jednostavno nije radila kako je očekivano. Tek nakon što sam metodom pokušaja i pogreške shvatio da Word interno ne čuva tekst onako kako ga vidimo, nego ga dijeli na segmente koji ovise o formatiranju i internim optimizacijama, rješenje je postalo jasno. Taj detalj nije nigdje eksplicitno naglašen u službenoj dokumentaciji pa je traženje uzroka oduzelo više vremena nego sama implementacija. S druge strane, generiranje Excel izvještaja s formulama i stilovima teklo je glatko, što sugerira kako je Excel podrška u POI-u zrelija i bolje dokumentirana od Word podrške.

Važno je napomenuti kako postoji ograničenje vezano uz kompatibilnost XDDF grafova s LibreOffice Calc aplikacijom. Iako je generirani XML graf ispravan i u potpunosti u skladu s OOXML standardom, što je potvrđeno pregledom internog XML sadržaja putem `getCTChart().xmlText()` metode, LibreOffice ne renderira stupce grafa ispravno te prikazuje samo prazne osi. Isti graf ispravno se prikazuje u Microsoft Excelu i Google Sheetsu. Ovaj problem je nekompatibilnost između POI-jevog XDDF API-ja i LibreOfficeovog parsera OOXML grafova te nije uzrokovan greškom u kodu. Za produkcijska okruženja koja zahtijevaju prikaz u LibreOfficeu, preporučuje se zaobilazanje XDDF API-ja i direktno upisivanje XML strukture grafa kroz `getCTChartSpace().set()` metodu.

Apache POI preporučuje se za sve scenarije koji zahtijevaju automatiziranu obradu Office dokumenata u Java okruženju, od generiranja izvještaja i faktura do masovne personalizacije dokumenata u obrazovnim i poslovnim sustavima.

# Popis isječaka koda

|     |                                                     |    |
|-----|-----------------------------------------------------|----|
| 1.  | Ovisnosti u datoteci pom.xml . . . . .              | 7  |
| 2.  | Obrada Word dokumenta . . . . .                     | 7  |
| 3.  | Kreiranje naslova . . . . .                         | 8  |
| 4.  | Kreiranje zaglavlja tablice . . . . .               | 8  |
| 5.  | Punjenje tablice podacima . . . . .                 | 9  |
| 6.  | Spremanje dokumenta . . . . .                       | 9  |
| 7.  | Datoteka studenti.csv . . . . .                     | 11 |
| 8.  | Kreiranje stilova ćelija . . . . .                  | 12 |
| 9.  | Popunjavanje tablice podacima . . . . .             | 13 |
| 10. | Unos formule i formatiranje retka . . . . .         | 13 |
| 11. | Automatska širina i zamrzavanje zaglavlja . . . . . | 13 |
| 12. | Kreiranje stupčastog grafa . . . . .                | 15 |
| 13. | Cirkularno pismo . . . . .                          | 23 |
| 14. | Excel izvještaj . . . . .                           | 25 |



# Popis literature

- [1] Apache Software Foundation, *Apache POI; Project History* — *poi.apache.org*, <https://poi.apache.org/devel/history/index.html>, [Pristupljeno 24. travnja 2026.]
- [2] Apache Software Foundation, *Apache POI - Wikipedia* — *en.wikipedia.org*, [https://en.wikipedia.org/wiki/{A}pache\\_{P}{O}{I}#{H}istory\\_and\\_roadmap](https://en.wikipedia.org/wiki/{A}pache_{P}{O}{I}#{H}istory_and_roadmap), [Pristupljeno 24. travnja 2026.]
- [3] Apache Software Foundation, *Apache POI-Component Overview* — *poi.apache.org*, <https://poi.apache.org/components/>, [Pristupljeno 25. travnja 2026.]
- [4] Apache Software Foundation, *Apache POI 5.5.1 API Documentation*, <https://poi.apache.org/apidocs/dev/>, [Pristupljeno 25. travnja 2026.]
- [5] Apache Software Foundation, *POI-HSSF and POI-XSSF/SXSSF - Java API To Access Microsoft Excel Format Files* — *poi.apache.org*, <https://poi.apache.org/components/spreadsheet/>, [Pristupljeno 29. travnja 2026.]
- [6] Apache Software Foundation, *Apache POI-HWPF and XWPF - Java API to Handle Microsoft Word Files*, <https://poi.apache.org/components/document/>, [Pristupljeno 29. travnja 2026.]
- [7] Apache Software Foundation, *Apache POI-HSLF - A Quick Guide* — *poi.apache.org*, <https://poi.apache.org/components/slideshow/quick-guide.html>, [Pristupljeno 29. travnja 2026.]
- [8] Apache Software Foundation, *Apache POI - Download Release Artifacts* — *poi.apache.org*, <https://poi.apache.org/download.html>, [Pristupljeno 05. svibnja 2026.]
- [9] Apache Software Foundation, *XWPF Quick Guide*, <https://poi.apache.org/components/document/quick-guide-xwpf.html>, [Pristupljeno 14. svibnja 2026.]
- [10] Apache Software Foundation, *HWPF and XWPF — Java API to Handle Microsoft Word Files*, <https://poi.apache.org/components/document/>, [Pristupljeno 14. svibnja 2026.]
- [11] Apache Software Foundation, *Busy Developer's Guide to HSSF and XSSF Features*, <https://poi.apache.org/components/spreadsheet/quick-guide.html>, [Pristupljeno 19. svibnja 2026.]

# Popis slika

|    |                                                               |    |
|----|---------------------------------------------------------------|----|
| 1. | Word dokument kreiran pomoću Apache POI biblioteke . . . . .  | 10 |
| 2. | Excel dokument kreiran pomoću Apache POI biblioteke . . . . . | 14 |
| 3. | Excel dokument kreiran pomoću Apache POI biblioteke . . . . . | 16 |
| 4. | Excel dokument kreiran pomoću Apache POI biblioteke . . . . . | 16 |

# Popis tablica

|                                        |   |
|----------------------------------------|---|
| 1. Pregled Apache POI modula . . . . . | 3 |
|----------------------------------------|---|

## **Prilozi**

# Prilog

## Cirkularno pismo

```
1 package com.zpracic.word;
2
3 import java.io.*;
4 import java.time.LocalDate;
5 import java.time.format.DateTimeFormatter;
6 import org.apache.poi.xwpf.usermodel.*;
7
8 public class CirkularnoPismo {
9
10     public static void main(String[] args) throws IOException {
11         String csvDatoteka = "src/main/java/com/zpracic/word/studenti.csv";
12         String predlozakDatoteka = "src/main/java/com/zpracic/word/
13             ↪ predlozak.docx";
14         String datum = LocalDate.now()
15             .format(DateTimeFormatter.ofPattern("dd.MM.yyyy."));
16
17         try (BufferedReader br = new BufferedReader(
18             new FileReader(csvDatoteka))) {
19             String linija;
20             boolean prviRed = true;
21             while ((linija = br.readLine()) != null) {
22                 if (prviRed) {
23                     prviRed = false;
24                     continue;
25                 }
26                 String[] podaci = linija.split(",");
27                 String ime = podaci[0];
28                 String prezime = podaci[1];
29                 String jmbag = podaci[2];
30                 String kolegi = podaci[3];
31                 String ocjena = podaci[4];
32
33                 try (FileInputStream fis = new FileInputStream(
34                     ↪ predlozakDatoteka);
35                     XWPFDocument dokument = new XWPFDocument(fis)) {
```

```

34
35         for (XWPFPParagraph paragraf : dokument.getParagraphs())
36             ↪ {
37                 zamijeniUParagrafu(paragraf, ime, prezime,
38                                     jmbag, kolegij, ocjena, datum);
39             }
40
41         String izlazDatoteka = "src/main/java/com/zpracic/word/
42             ↪ obavijest_"
43             + ime + "_" + prezime + ".docx";
44         try (FileOutputStream out =
45             new FileOutputStream(izlazDatoteka)) {
46             dokument.write(out);
47         }
48         System.out.println("Kreiran: " + izlazDatoteka);
49     }
50     System.out.println("Cirkularno pismo uspješno generirano.");
51 }
52
53 private static void zamijeniUParagrafu(XWPFPParagraph paragraf,
54     String ime, String prezime, String jmbag,
55     String kolegij, String ocjena, String datum) {
56
57     StringBuilder cijeli = new StringBuilder();
58     for (XWPFRun run : paragraf.getRuns()) {
59         if (run.getText(0) != null) {
60             cijeli.append(run.getText(0));
61         }
62     }
63     String tekst = cijeli.toString();
64     tekst = tekst.replace("{{IME}}", ime);
65     tekst = tekst.replace("{{PREZIME}}", prezime);
66     tekst = tekst.replace("{{JMBAG}}", jmbag);
67     tekst = tekst.replace("{{KOLEGIJ}}", kolegij);
68     tekst = tekst.replace("{{OCJENA}}", ocjena);
69     tekst = tekst.replace("{{DATUM}}", datum);
70
71     if (!paragraf.getRuns().isEmpty()) {
72         paragraf.getRuns().get(0).setText(tekst, 0);
73         for (int i = 1; i < paragraf.getRuns().size(); i++) {
74             paragraf.getRuns().get(i).setText("", 0);
75         }
76     }
77 }
78 }

```

# Excel izvještaj

```
1 package com.zpracic.excel;
2
3 import java.io.FileOutputStream;
4 import java.io.IOException;
5
6 import org.apache.poi.ss.usermodel.*;
7 import org.apache.poi.xssf.usermodel.*;
8
9 import org.apache.poi.xddf.usermodel.chart.*;
10 import org.apache.poi.ss.util.CellRangeAddress;
11 import org.apache.poi.xssf.usermodel.XSSFChart;
12 import org.apache.poi.xssf.usermodel.XSSFClientAnchor;
13 import org.apache.poi.xssf.usermodel.XSSFDrawing;
14
15 public class Excel {
16
17     public static void main(String[] args) throws IOException {
18
19         XSSFWorkbook radnaKnjiga = new XSSFWorkbook();
20         // Dodavanje novog lista (sheet-a) u radnu knjigu
21         XSSFSheet list = radnaKnjiga.createSheet("Ocjene studenata");
22
23         XSSFCellStyle stilZaglavljja = radnaKnjiga.createCellStyle();
24         stilZaglavljja.setFillForegroundColor(new XSSFColor(
25             new java.awt.Color(46, 117, 182), new
26             ↪ DefaultIndexedColorMap()));
27         stilZaglavljja.setFillPattern(FillPatternType.SOLID_FOREGROUND);
28         stilZaglavljja.setAlignment(HorizontalAlignment.CENTER);
29         stilZaglavljja.setBorderBottom(BorderStyle.THIN);
30         XSSFFont fontZaglavljja = radnaKnjiga.createFont();
31         fontZaglavljja.setBold(true);
32         fontZaglavljja.setColor(new XSSFColor(
33             new java.awt.Color(255, 255, 255), new
34             ↪ DefaultIndexedColorMap()));
35         fontZaglavljja.setFontHeightInPoints((short) 11);
36         stilZaglavljja.setFont(fontZaglavljja);
37
38         XSSFCellStyle stilParni = radnaKnjiga.createCellStyle();
39         stilParni.setAlignment(HorizontalAlignment.CENTER);
40
41         XSSFCellStyle stilNeparni = radnaKnjiga.createCellStyle();
42         stilNeparni.setFillForegroundColor(new XSSFColor(
43             new java.awt.Color(222, 234, 246), new
44             ↪ DefaultIndexedColorMap()));
45         stilNeparni.setFillPattern(FillPatternType.SOLID_FOREGROUND);
46         stilNeparni.setAlignment(HorizontalAlignment.CENTER);
```

```

44
45 XSSFCellStyle stilZbroj = radnaKnjiga.createCellStyle();
46 stilZbroj.setAlignment(HorizontalAlignment.CENTER);
47 stilZbroj.setBorderTop(BorderStyle.MEDIUM);
48 XSSFFont fontZbroj = radnaKnjiga.createFont();
49 fontZbroj.setBold(true);
50 stilZbroj.setFont(fontZbroj);
51
52
53 String[] stupci = {"Ime i prezime", "JMBAG", "Kolegij",
54                   "Ocjena (broj)", "Ocjena (opis)"};
55 Row redZaglavljaja = list.createRow(0);
56 for (int i = 0; i < stupci.length; i++) {
57     Cell celija = redZaglavljaja.createCell(i);
58     celija.setCellValue(stupci[i]);
59     celija.setCellStyle(stilZaglavljaja);
60 }
61
62 String[][] podaci = {
63     {"Pero Kos", "0123456789", "Napredne WEB tehnologije", "5",
64      ↪ "Izvrstan"},
65     {"Franjo Kovac", "0123456790", "Napredne WEB tehnologije", "4",
66      ↪ "Vrlo dobar"},
67     {"Maja Peric", "0123456791", "Napredne WEB tehnologije", "5",
68      ↪ "Izvrstan"},
69     {"Luka Novak", "0123456792", "Napredne WEB tehnologije", "3",
70      ↪ "Dobar"},
71     {"Sara Babic", "0123456793", "Napredne WEB tehnologije", "4",
72      ↪ "Vrlo dobar"},
73 };
74
75 for (int i = 0; i < podaci.length; i++) {
76     Row red = list.createRow(i + 1);
77     XSSFCellStyle stil = (i % 2 == 0) ? stilParni : stilNeparni;
78     for (int j = 0; j < podaci[i].length; j++) {
79         Cell celija = red.createCell(j);
80         if (j == 3) {
81             celija.setCellValue(Double.parseDouble(podaci[i][j]));
82         } else {
83             celija.setCellValue(podaci[i][j]);
84         }
85         celija.setCellStyle(stil);
86     }
87 }
88
89 int zadnjiRed = podaci.length + 1;
90 Row redFormule = list.createRow(zadnjiRed);
91
92 Cell celijaOznaka = redFormule.createCell(2);

```



```

88     celijaOznaka.setCellValue("Prosjek ocjena:");
89     celijaOznaka.setCellStyle(stilZbroj);
90
91     Cell celijaFormula = redFormule.createCell(3);
92     celijaFormula.setCellFormula("AVERAGE(D2:D6)");
93     celijaFormula.setCellStyle(stilZbroj);
94
95     for (int i = 0; i < stupci.length; i++) {
96         list.autoSizeColumn(i);
97     }
98     list.createFreezePane(0, 1);
99
100    XSSFFormulaEvaluator evaluator = radnaKnjiga.getCreationHelper()
101        .createFormulaEvaluator();
102    evaluator.evaluateAll();
103
104
105    XSSFDrawing drawing = list.createDrawingPatriarch();
106    XSSFClientAnchor anchor = drawing.createAnchor(0, 0, 0, 0, 0, 8, 6,
107        ↪ 25);
108
109    XSSFChart chart = drawing.createChart(anchor);
110    chart.setTitleText("Ocjene studenata");
111    chart.setTitleOverlay(false);
112
113    XDDFChartLegend legenda = chart.getOrAddLegend();
114    legenda.setPosition(LegendPosition.BOTTOM);
115
116    XDDFCategoryAxis xOs = chart.createCategoryAxis(AxisPosition.BOTTOM
117        ↪ );
118    xOs.setTitle("Student");
119    XDDFValueAxis yOs = chart.createValueAxis(AxisPosition.LEFT);
120    yOs.setTitle("Ocjena");
121    yOs.setCrosses(AxisCrosses.AUTO_ZERO);
122
123    XDDFDataSource<String> imena = XDDFDataSourcesFactory.
124        ↪ fromStringCellRange(list,
125        ↪ CellRangeAddress.valueOf("A2:A6"));
126
127    XDDFNumericalDataSource<Double> ocjene = XDDFDataSourcesFactory
128        ↪ .fromNumericCellRange(list, CellRangeAddress.valueOf("D2:D6
129        ↪ "));
130
131    XDDFChartData podaciGrafica = chart.createData(ChartTypes.BAR, xOs,
132        ↪ yOs);
133    podaciGrafica.setVaryColors(true); // Svaki stupac dobiva čdrugaiju
134        ↪ boju
135
136    XDDFChartData.Series serija = podaciGrafica.addSeries(imena, ocjene);

```

```

131     serija.setTitle("Ocjena", null);
132
133     ((XDDFBarChartData) podaciGrafa).setBarDirection(BarDirection.COL);
134
135     chart.plot(podaciGrafa);
136
137     XSSFDrawing drawingPie = list.createDrawingPatriarch();
138     XSSFClientAnchor anchorPie = drawingPie.createAnchor(0, 0, 0, 0, 7,
139         ↪ 8, 13, 25);
139
140     XSSFChart pieChart = drawingPie.createChart(anchorPie);
141     pieChart.setTitleText("Raspodjela ocjena (%)");
142     pieChart.setTitleOverlay(false);
143
144     XDDFChartLegend legendaPie = pieChart.getOrAddLegend();
145     legendaPie.setPosition(LegendPosition.BOTTOM);
146
147     XSSFSheet pomocniList = radnaKnjiga.createSheet("Podaci grafa");
148
149     String[] ocjeneOpis = {"Dobar (3)", "Vrlo dobar (4)", "Izvrstan (5)
150         ↪ "};
151     double[] brojevi = {1, 2, 2}; // 1x ocjena 3, 2x ocjena 4, 2x
152         ↪ ocjena 5
153
154     for (int i = 0; i < ocjeneOpis.length; i++) {
155         Row r = pomocniList.createRow(i);
156         r.createCell(0).setCellValue(ocjeneOpis[i]);
157         r.createCell(1).setCellValue(brojevi[i]);
158     }
159
160     XDDFDataSource<String> kategorije = XDDFDataSourcesFactory.
161         ↪ fromStringCellRange(
162         ↪     pomocniList, CellRangeAddress.valueOf("A1:A3"));
163     XDDFNumericalDataSource<Double> vrijednosti =
164         ↪ XDDFDataSourcesFactory
165         ↪     .fromNumericCellRange(pomocniList, CellRangeAddress.valueOf
166         ↪     ↪ ("B1:B3"));
167
168     XDDFChartData pieData = pieChart.createData(ChartTypes.PIE, null,
169         ↪ null);
170     pieData.setVaryColors(true);
171     pieData.addSeries(kategorije, vrijednosti);
172
173     pieChart.plot(pieData);
174
175     try (FileOutputStream out = new FileOutputStream(
176         ↪ "src/main/java/com/zpracic/excel/ocjene.xlsx")) {
177         radnaKnjiga.write(out);
178     }

```

```
173         radnaKnjiga.close();
174         System.out.println("Excel datoteka uspjesno kreirana: ocjene.xlsx")
           ↵ ;
175     }
176 }
```

---

Isječak koda 14: Excel izvještaj